

Supplementary Material (Multi-Agent Dual-Path Verification Framework for Scalable Graph Reasoning)

A Per-Benchmark Detailed Results

The following tables disaggregate the main-paper accuracies by task category. The “Previous SOTA” column follows the strongest comparison values used in the corresponding benchmark-level analysis. Brief observations highlight the main gains and residual failure modes.

A.1 GraphWiz

Table 1: Task-level accuracy (%) on GraphWiz.

Task	GPT	Previous SOTA	DuVerG
Cycle	82.0	100.0	99.2
Connect	64.8	86.5	100.0
Bipartite	67.3	96.3	100.0
Topology	100.0	100.0	95.0
Shortest	8.5	95.0	100.0
Triangle	10.3	94.0	89.2
Flow	19.3	94.0	97.5
Hamilton	36.3	32.5	99.2
Subgraph	40.3	99.3	100.0
Avg	47.6	88.6	97.8

Observation. Gains are strongest on shortest path, flow, Hamiltonian, and subgraph queries, while cycle, topology, and triangle counting retain visible headroom.

A.2 NLGraph

Table 2: Task-level accuracy (%) on NLGraph.

Task	GPT	Previous SOTA	DuVerG
Connect	72.0	97.0	100.0
Cycle	52.9	100.0	100.0
Topology	20.0	94.8	97.8
SP	35.9	98.4	100.0
MaxFlow	98.3	100.0	98.3
Bipartite	29.8	100.0	98.8
Hamilton	22.4	100.0	100.0
GNN	0	97.4	100.0
Avg	51.4	97.8	99.5

Observation. NLGraph is close to saturation for stronger systems; the remaining margin mainly reflects robustness across task formulations.

A.3 GraphInstruct

Table 3: Task-level accuracy (%) on GraphInstruct.

Task	GPT	Previous SOTA	DuVerG
Neighbor	98.0	100.0	99.0
Bipartite	89.7	99.0	100.0
Edge	90.7	99.3	94.7
Connectivity	92.2	100.0	99.7
Degree	98.0	100.0	99.7
DFS	89.3	95.7	97.0
Predecessor	47.4	98.0	99.7
TopoSort	59.0	98.3	99.7
ConnComp	39.3	94.7	99.7
CommonNbr	49.0	100.0	99.0
Hamilton	33.7	97.7	100.0
Jaccard	17.0	100.0	99.0
SP	57.3	100.0	100.0
Diameter	23.0	100.0	99.7
PageRank	15.3	16.3	91.7
MaxFlow	15.0	99.3	100.0
MST	1.0	0	100.0
Avg	53.7	88.1	98.7

Observation. The largest gains appear on PageRank and MST; near-ceiling structural tasks show much smaller differences.

A.4 GraphArena

Table 4: Task-level accuracy (%) on GraphArena.

Task	GPT	Previous SOTA	DuVerG
Connect	56.1	100.0	95.1
Diameter	32.3	0.0	95.0
Distance	61.8	92.4	99.3
GED	20.0	3.1	38.0
MCP	29.6	98.0	99.7
MCS	50.7	12.7	86.0
MIS	26.9	68.3	90.6
MVC	14.7	9.7	49.3
Neighbor	82.0	98.8	99.8
TSP	14.3	23.0	63.7
Avg	39.2	50.6	83.8

Observation. GraphArena shows strong gains on several combinatorial categories, while GED, MVC, and TSP remain the main residual difficulties.

A.5 OGB Benchmarks

Table 5: Task-level accuracy (%) on the OGB-derived benchmarks.

Dataset / Task	GPT	Previous SOTA	DuVerG
<i>ogbn-arxiv</i>			
TriCount	13.5	99.2	100.0
ClustCoef	14.6	83.5	100.0
3-Reach	44.2	99.3	98.1
PageRank	15.5	30.3	76.2
Avg	22.4	79.0	93.5
<i>ogbl-ppa</i>			
TriCount	3.2	100.0	100.0
ClustCoef	3.8	88.0	100.0
3-Reach	30.9	85.4	83.8
PageRank	3.2	21.1	47.8
Avg	10.5	74.6	82.6

Observation. Projection and symbolic execution are strongest on local counting and clustering; 3-hop reachability slightly favors the previous SOTA, and local PageRank remains the hardest OGB task.

B Reproducibility and Implementation Details

These control/resource constants were selected on preliminary development queries before test evaluation and then fixed across benchmarks.

B.1 Fixed Hyperparameters

Table 6: Fixed control and resource settings used by DuVerG.

Category	Hyperparameter	Value
Routing	Large-graph threshold θ_{large}	2000
	Density threshold θ_{ρ}	0.05
	Automatic projection scale	2000
	trigger θ_{auto}	
LPSP	Default hop k	2
	Min. node budget $B_{\text{v}}^{\min}$	2000
	Max. node budget $B_{\text{v}}^{\max}$	2×10^6
	Max. edge budget $B_{\text{e}}^{\max}$	10^7
	Base branching factor b_{base}	50
LLM	Router / Planner / Critic	0.1
	temperature	
	Coder A / B temperature	0.2
	Reasoner T_1	0.2
Exec.	Reasoner T_2	0.6
	Sandbox timeout	300 s
	Task timeout	300 s
	Max. repair iterations M	2

Routing thresholds protect path selection, LPSP bounds the retained view, temperatures induce controlled diversity, and execution limits bound recovery cost. No row is tuned separately for ogbn-arxiv or ogbl-ppa.

B.2 Streaming LPSP Semantics

For large on-disk graphs, LPSP preserves the in-memory seed, depth, node-budget, and edge-budget semantics without materializing the full graph as a NetworkX object. It maintains a validated multi-source frontier, scans only adjacency relevant to the current frontier, deduplicates new nodes before admission, checks remaining capacity at each level, and finally collects induced edges under the edge cap.

1. **Validate anchors.** Keep only query-mentioned seed identifiers that occur in the graph. If no valid anchor remains, projection is bypassed rather than replaced by a hub-centered surrogate.
2. **Expand under a hard node bound.** Initialize one joint frontier from all valid seeds and expand level by level to depth k , admitting at most the remaining capacity under B_v .
3. **Construct the retained edge view.** Scan for edges whose endpoints lie in the visited set; if more than B_e are induced, apply the implementation edge score and retain at most B_e edges.
4. **Record budget status.** The shared state records whether the depth, node bound, or edge bound became active so that downstream verification can distinguish a complete local view from a resource-truncated one.

For global queries, a missing anchor never triggers a hub fallback: the full-graph path is retained when feasible, otherwise the resource limitation is surfaced.

B.3 Verification and Repair Semantics

Verification is status-aware: normalized agreement permits early exit without certification, whereas disagreement between successful executions triggers a targeted check over the existing traces. Exceptions and timeouts enter at most M repair rounds; persistent failure may switch paths while retaining parsed query and projection state. No third solver is introduced.

C Prompt Templates

The templates below are populated from shared state at inference time and contain no benchmark answers or solution traces. Paired-agent diversity is stated explicitly.

C.1 Router Agent

The Router receives the query with lightweight graph metadata and emits structured control information that is validated before projection or path execution.

Prompt Template

```
system: "You are a query router for a hybrid neural-
symbolic graph reasoning system."
user_template: |
  Query: {query}
  TaskType: {task_type}
  GraphStats: {graph_stats}

  Decide route:
```

- symbolic: pick this when the query asks for a computation that has a precise procedural answer (path, flow, matching, substructure, traversal, counting). The system will generate and execute code.
- neural: pick this when the query asks for an interpretation, preference, or attribute-based judgment that does not reduce to a single procedure. The system will reason directly.

Output JSON:

```
{
  "route": "symbolic" or "neural",
  "why": "Brief reasoning",
  "need_decomposition": true/false,
  "decomposition": {
    "seed_nodes": [],
    "hop": {default_hop},
    "node_budget": {node_budget},
    "edge_budget": {edge_budget}
  }
}
```

C.2 Planner Agent

The Planner is used only on the symbolic path. It specifies an algorithm family, key steps, and defensive checks without itself producing executable code.

Prompt Template

```
system: "You are a graph task planner. Analyze the
problem and outline logical steps for an
implementation."
user_template: |
  Task: {query}
  TaskType: {task_type}
  GraphStats: {graph_stats}
  GraphDataHint: {graph_hint}

  Provide a high-level execution plan:
  - algorithm_family: the category of algorithm (e.g.,
    BFS, flow, backtracking, isomorphism).
  - key_steps: logical steps to solve the problem
    programmatically.
  - defensive_checks: edge cases to handle (disconnected
    graph, missing nodes, type mismatch).

  Output JSON:
  {
    "algorithm_family": "...",
    "key_steps": ["..."],
    "defensive_checks": ["..."]
  }
```

C.3 Coder Agents

The two Coders share an input schema but differ in implementation style: Coder A favors canonical NetworkX primitives, whereas Coder B favors explicit implementations with visible corner-case checks. This creates operational diversity without assuming independence.

Prompt Template

```
system: "You are an expert Python programmer
specializing in graph algorithms. Write robust,
fault-tolerant code."
user_template: |
  Task: {query}
  TaskType: {task_type}
  Plan: {plan_json}
  ImplementationStyle: {implementation_style}

  Graph Input Data:
  {graph_payload_hint}
```

Constraints:

1. Use only standard Python libraries and networkx.
2. Do not install new packages.
3. Print only the final answer. Do not print debug info.
4. Handle missing nodes, disconnected graphs, and type mismatches explicitly.

If this is a retry, fix the execution error:
{error_feedback}

Write the complete Python code in a single “python” block.

implementation_style(A):

Prefer canonical NetworkX primitives when a direct, well-defined routine exists.

implementation_style(B):

Prefer an explicit implementation of the graph operation and make corner-case checks visible in the code.

C.4 Reasoner Agents

The neural pair shares a prompt and graph view. Fixed temperatures reported in Appendix B pair a conservative completion with a more exploratory one.

Prompt Template

system: "You are a logical graph reasoner. Solve the problem step-by-step using semantic reasoning."

user_template: |
Task: {query}
TaskType: {task_type}
GraphSummary: {graph_hint}

Perform chain-of-thought reasoning and provide the final answer.
Answer:

C.5 Critic Agent

The Critic receives candidate outputs together with execution status, performs answer-type-aware normalization, and separates valid negative/empty answers from execution failures. Agreement allows early exit without certification; successful disagreement requests a concrete check.

Prompt Template

system: "You are a strict execution-grounded verifier. Compare two candidate outputs under the query’s answer type and execution status."

consistency_template: |
Query: {query}
AnswerType: {answer_type}
StatusA: {status_a}
OutputA: {out_a}
StatusB: {status_b}
OutputB: {out_b}

Prompt Template (continued)

Rules:

1. Normalize only format differences that preserve the task answer (case, whitespace, Boolean aliases, set ordering, and benchmark-specified numeric tolerance).
2. If both successful outputs are equivalent after normalization, return consistent=true.
3. If exactly one execution failed or timed out and the other produced a well-formed answer, prefer the successful candidate.
4. Never interpret a legitimate "No"/"False"/empty answer as failure solely from its text.
5. If both executions succeed but disagree, identify one concrete condition that distinguishes the existing candidates and request a focused check.

Output JSON:

```
{{  
  "consistent": true/false,  
  "why": "Brief explanation",  
  "resolution": "pick_a" or "pick_b" or "  
    need_tiebreaker",  
  "tiebreaker_hint": "A specific condition to check  
    against the two existing candidates."  
}}
```

Focused Tiebreaker Check

The tiebreaker verifies the two existing traces rather than generating a third solution. It checks only the concrete condition identified from the disagreement.

Prompt Template

focused_check_template: |
Query: {query}
CandidateA: {out_a}
CandidateB: {out_b}
Check: {tiebreaker_hint}
RelevantTraceA: {trace_a}
RelevantTraceB: {trace_b}

Evaluate only the requested condition against the two existing candidates.
Do not generate a third solution.
Return: pick_a, pick_b, or unresolved.

Status and Fallback Handling

Execution status is separate from answer content, so “No,” “False,” or an empty set remain valid outputs. Exceptions/-timeouts use the bounded repair loop; unresolved conflicts trigger fallback. Normalization handles format variation, status handles execution failure, and the focused check handles semantic disagreement.